

实验：知识图谱表示学习模型 TransE

1. 安装 Anaconda (已安装跳过此步骤)

(1) 下载 Anaconda 并安装

清华镜像网：

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/>

选择下载最新版本（如当前最新版本为 2024.10-1 版本）

ANACONDA3 2024.10-1 (64-bit) Setup	Size	Time
Anaconda3-2024.10-1-MacOSX-arm64.pkg	744.6 MiB	2025-02-14 03:35
Anaconda3-2024.10-1-MacOSX-arm64.sh	747.2 MiB	2025-02-14 03:36
Anaconda3-2024.10-1-MacOSX-x86_64.pkg	776.0 MiB	2025-02-14 03:36
Anaconda3-2024.10-1-MacOSX-x86_64.sh	778.5 MiB	2025-02-14 03:36
Anaconda3-2024.10-1-Windows-x86_64.exe	950.5 MiB	2025-02-14 03:36
Anaconda3-4.0.0-Linux-x86.sh	336.9 MiB	2025-02-14 03:36
Anaconda3-4.0.0-Linux-x86_64.sh	398.4 MiB	2025-02-14 03:36
Anaconda3-4.0.0-MacOSX-x86_64.pkg	341.5 MiB	2025-02-14 03:36
Anaconda3-4.0.0-MacOSX-x86_64.sh	292.7 MiB	2025-02-14 03:36

Anaconda3 2024.10-1 (64-bit) Setup

Welcome to Anaconda3 2024.10-1 (64-bit) Setup

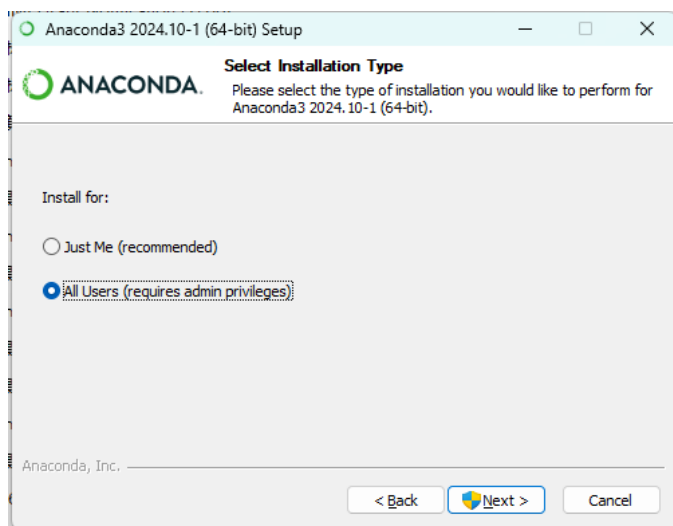
Setup will guide you through the installation of Anaconda3 2024.10-1 (64-bit).

It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.

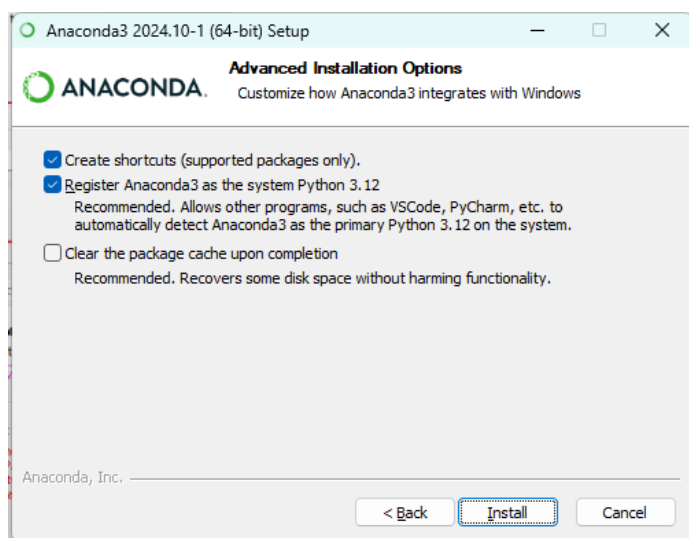
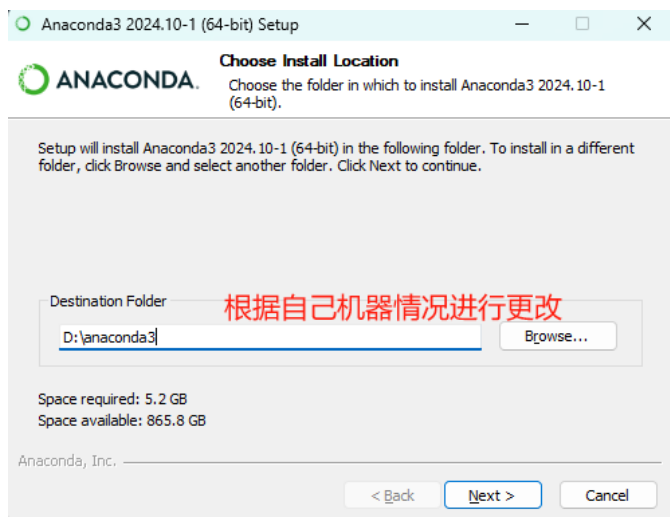
Click Next to continue.

Next > Cancel

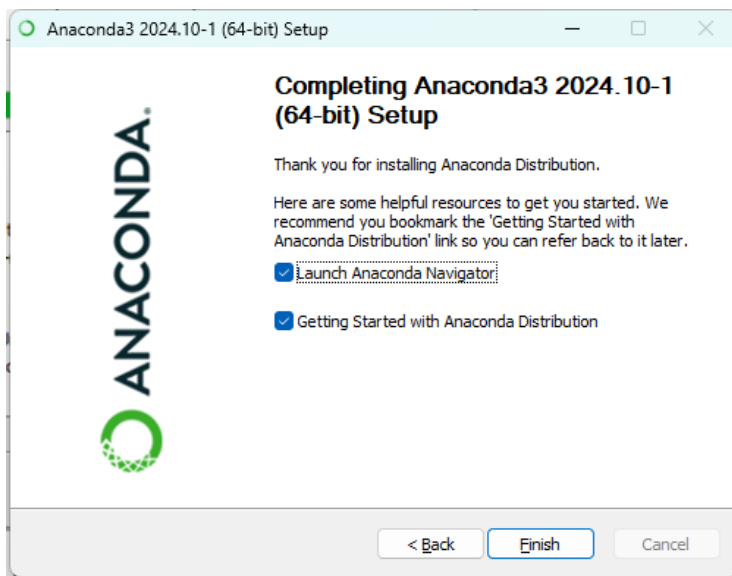
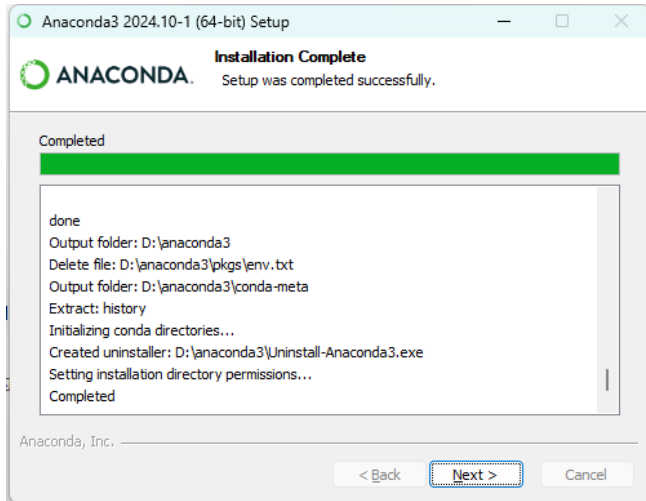
前面我们一直确认 next 即可。直到这里我们选择 all user。



这里选择安装路径，这里最好选择自己的路径（默认安装是安装在 C 盘）。



然后等待安装即可，安装文件有几 GB，时间会可能会比较长，因各自电脑配置而异，耐心等待即可。



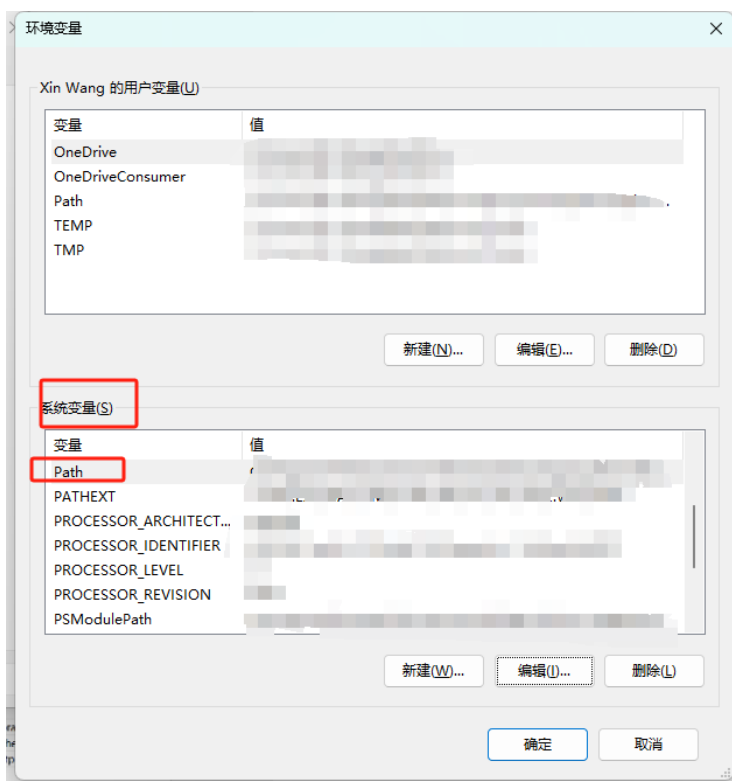
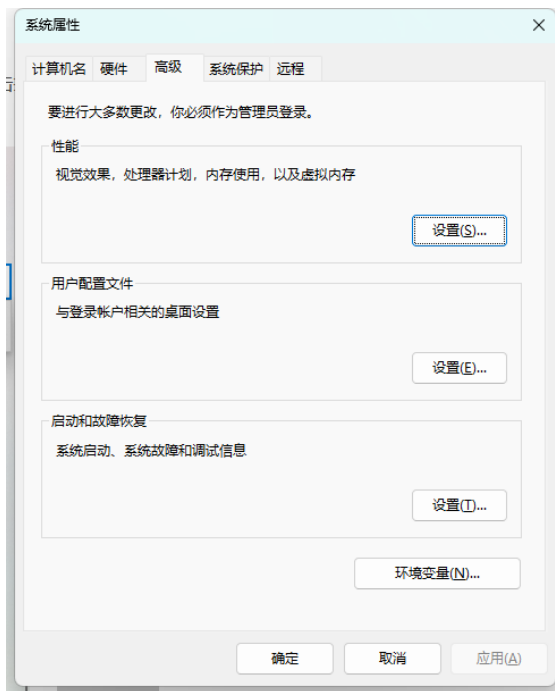
到这里 anaconda 已经安装完成。

(2) 配置环境变量

打开设置，搜索“查看高级系统设置”，点击打开。

若安装时已勾选自动配置，此步可忽略

打开环境变量。



双击打开，新建。

新建五个变量进去，将下面的五个变量的结合你的 `anaconda` 实际安装目录来更改写入。

(这里的 `anaconda` 安装路径为 `D:\anaconda3`，把下面的更改为你的 `anaconda` 路径即可)

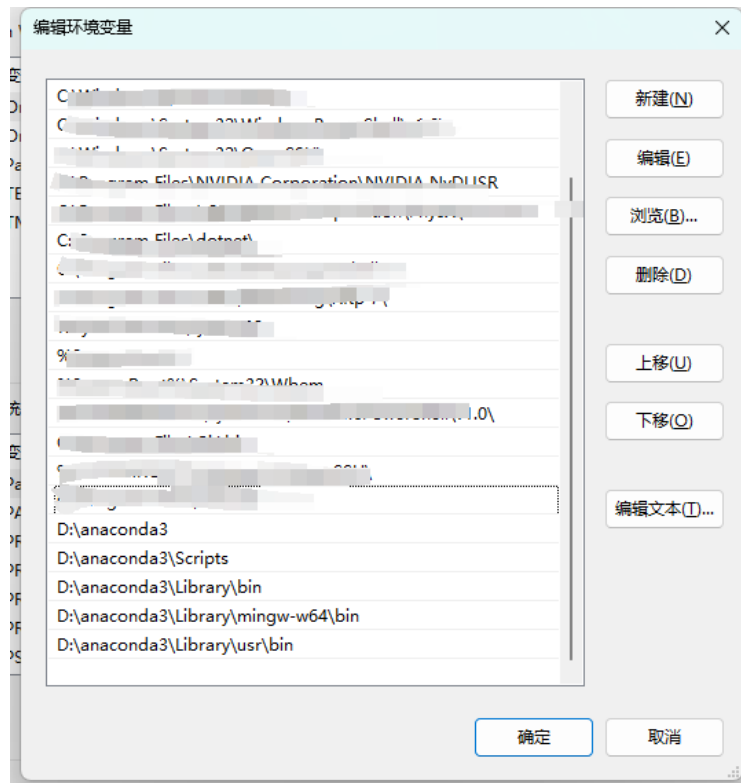
`D:\anaconda3`

`D:\anaconda3\Scripts`

`D:\anaconda3\Library\bin`

D:\anaconda3\Library\mingw-w64\bin

D:\anaconda3\Library\usr\bin

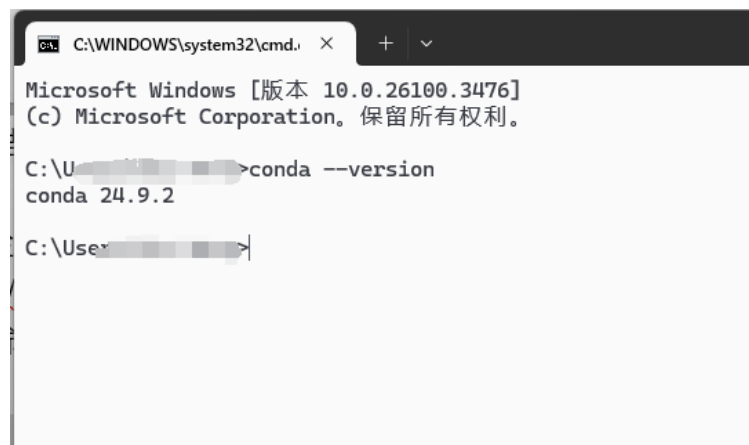


添加结束，确认退出。

(3) 检查是否安装成功

按下 Win+R，输入 cmd 打开终端。

输入命令检验。//检验 anaconda 版本



```
C:\Users\...>python
Python 3.12.7 | packaged by Anaconda, Inc. | (main, Oct 4 2024, 13:17:27) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> quit()
```

(4) 更改镜像源

(切记，更改镜像源需要在系统环境，如果第四步进入了 python 环境，需要先输入 exit

退出，或者重新打开一个终端更改镜像源)

cmd 后依次输入下面命令

直接输入以下命令即可，将默认的国外站点更改为国内的镜像源，速度更快!

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/
```

//设置搜索时显示通道地址

```
conda config --set show_channel_urls yes
```

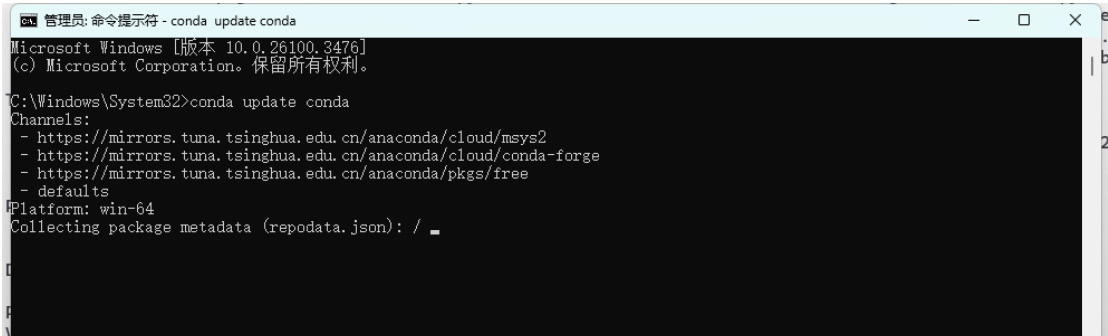
```
C:\User\>conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
C:\User\>conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
C:\User\>conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/
C:\User\>conda config --set show_channel_urls yes
C:\User\>
```

(5) 更新包 (需要较长时间，实验时建议跳过此步，有空余时间再更新)

更新时间较长，建议找个空余时间更新,不更新也可以，但为避免后续安装其他东西出错最好更一下

先更新 conda

```
conda update conda
```



```
管理员: 命令提示符 - conda update conda
Microsoft Windows [版本 10.0.26100.3476]
(c) Microsoft Corporation。保留所有权利。

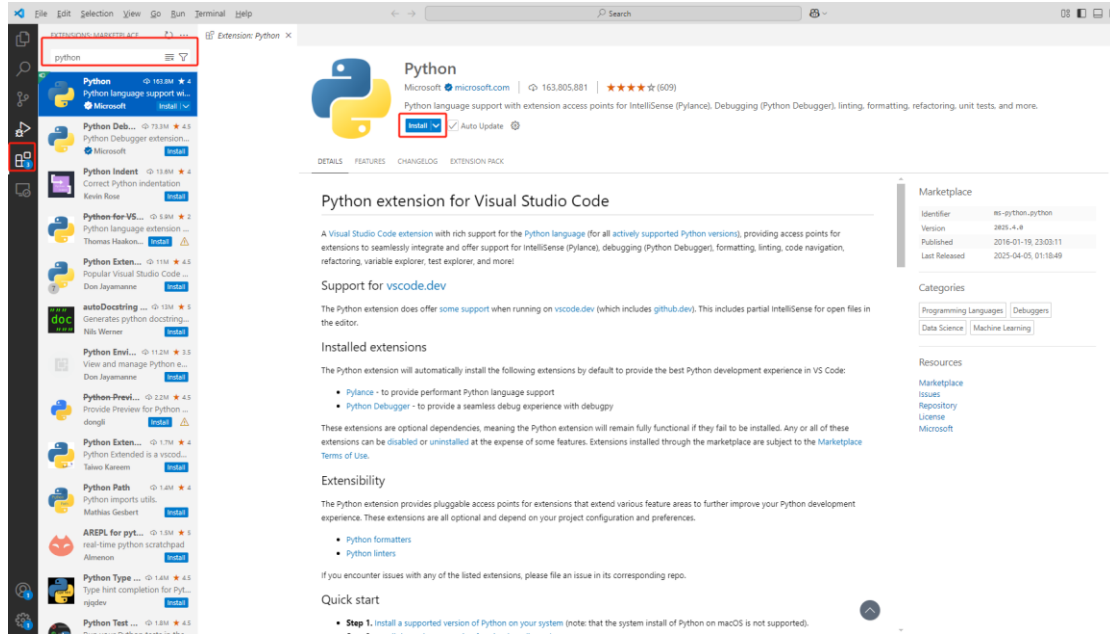
C:\Windows\System32>conda update conda

Channels:
- https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2
- https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
- https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
- defaults
Platform: win-64
Collecting package metadata (repodata.json): /
```

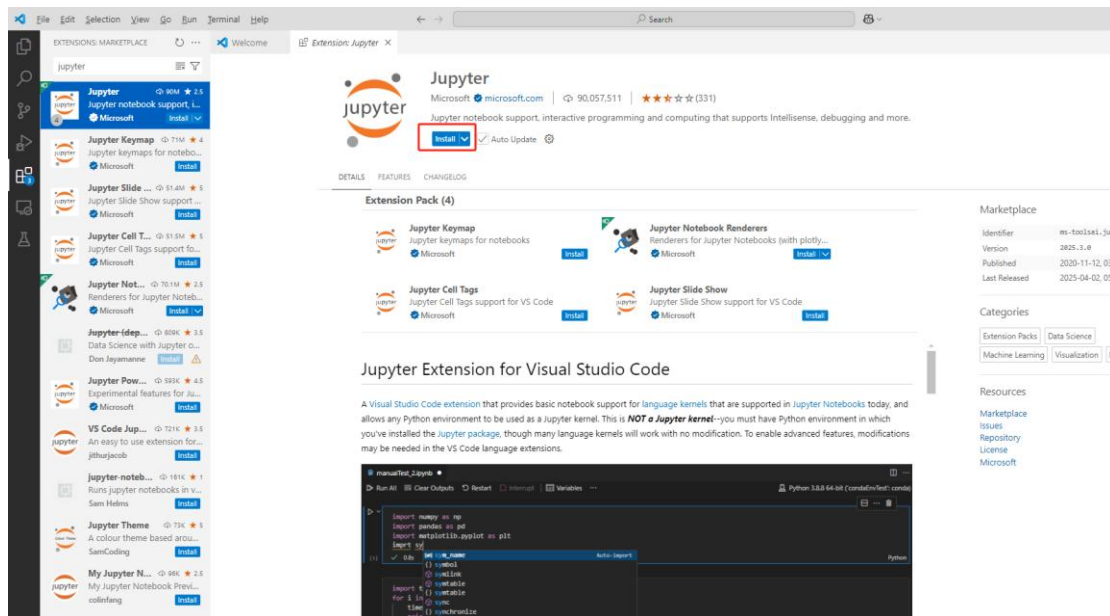
再更新第三方所有包

2. 配置 VS Code

(1) 安装 Python 插件

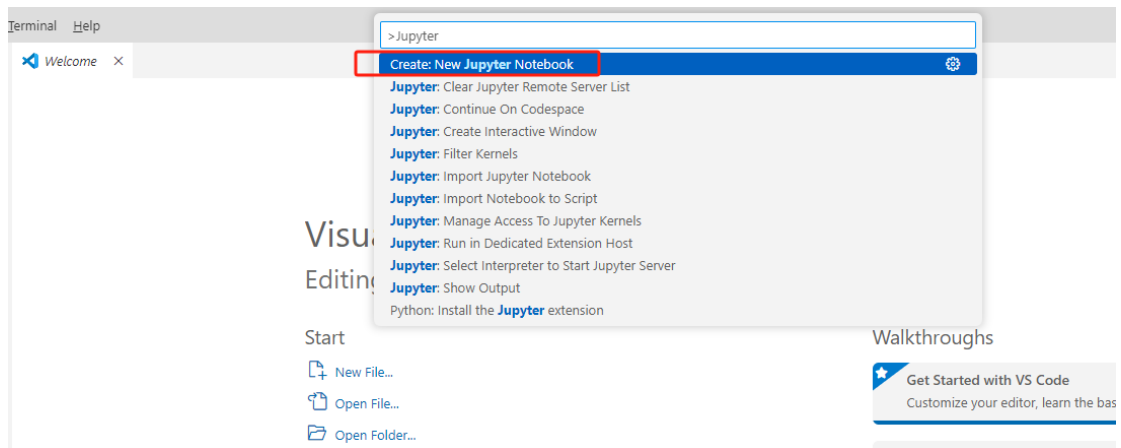


(2) 安装 Jupyter 插件

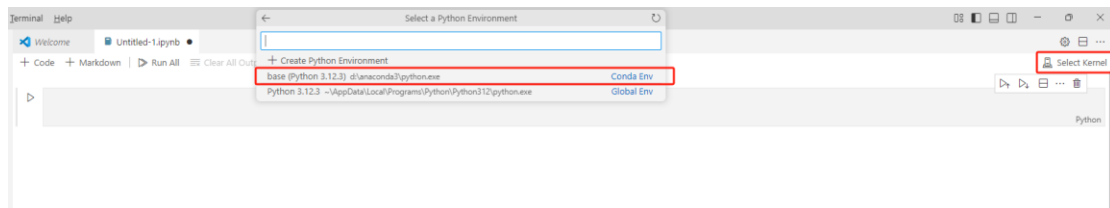


3. 在 VS Code 中使用 Jupyter Notebook

(1) 新建 Jupyter Notebook 文件



(2) 选择 Kernel



4. 实验原理与步骤

(1) 目的

知识图谱补全是从已知的知识图谱中提取出三元组(h,r,t)，为实体和关系进行建模，通过训练出的模型进行链接预测，以达成知识图谱补全的目标。

本文实验采用了 FB15K-237 数据集，分为训练集和测试集。利用训练集进行 transE 建模，通过训练为每个实体和关系建立起向量映射，并在测试集中计算 MeanRank 和 Hit10 指标进行结果检验。

(2) 数据集介绍

使用 FB15K-237 数据集

分为以下四个文件

- **entity2id.txt**

实体和 id 对


```

/m/06rf7    0
/m/0c94fn   1
/m/016ywr   2
/m/01yjl    3
/m/02hrh1q  4

```

- **relation2id.txt**

关系和 id 对

```

/people/appointed_role/appointment./people/appointment/appointed_by
/location/statistical_region/rent50_2./measurement_unit/dated_money_v
/tv/tv_series_episode/guest_stars./tv/tv_guest_role/actor    2
/music/performance_role/track_performances./music/track_contribution

```

- **train.txt**

训练集三元组（实体，实体，关系）

```

/m/027rn    /m/06cx9    /location/country/form_of_government
/m/017dcd    /m/06v8s0    /tv/tv_program/regular_cast./tv/regular_tv_a
/m/07s9rl0    /m/0170z3    /media_common/netflix_genre/titles
/m/01sl1q    /m/044mz_    /award/award_winner/awards_won./award/award
/m/0cnk2q    /m/02nzb8    /soccer/football_team/current_roster./sports

```

- **test.txt**

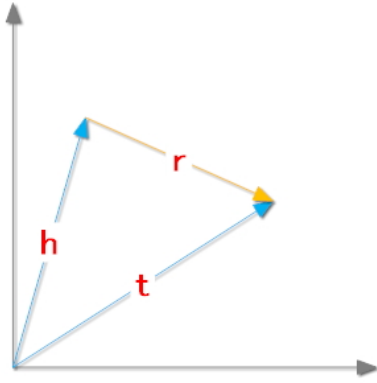
测试集三元组（实体，实体，关系）

(3) 方法

TransE 模型原理：

TransE 将起始实体，关系，指向实体映射成同一空间的向量，如果 (head, relation, tail)

存在，那么 $h + r \approx t$



目标函数为：

$$f_r(h, t) = \|h + r - t\|_2^2$$

算法：

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L , margin γ , embeddings dim. k .

1: **initialize** $\ell \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each $\ell \in L$

2: $\ell \leftarrow \ell / \|\ell\|$ for each $\ell \in L$

3: $e \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each entity $e \in E$

4: **loop**

5: $e \leftarrow e / \|e\|$ for each entity $e \in E$

6: $S_{batch} \leftarrow \text{sample}(S, b)$ // sample a minibatch of size b

7: $T_{batch} \leftarrow \emptyset$ // initialize the set of pairs of triplets

8: **for** $(h, \ell, t) \in S_{batch}$ **do**

9: $(h', \ell, t') \leftarrow \text{sample}(S'_{(h, \ell, t)})$ // sample a corrupted triplet

10: $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$

11: **end for**

12: Update embeddings w.r.t.
$$\sum_{((h, \ell, t), (h', \ell, t')) \in T_{batch}} \nabla [\gamma + d(h + \ell, t) - d(h' + \ell, t')]_+$$

13: **end loop**

a) 初始化

根据维度，为每个实体和关系初始化向量，并归一化

```

17
18         for relation in self.relation:
19             r_emb_temp = np.random.uniform(-6 / math.sqrt(self.embedding_dim),
20                                             6 / math.sqrt(self.embedding_dim),
21                                             self.embedding_dim)
22             relation_dict[relation] = r_emb_temp / np.linalg.norm(r_emb_temp, ord=2)
23

```

b) 选取 batch

设置 nbatches 为 batch 数目，batch_size = len(self.triple_list) // nbatches

从训练集中随机选择 batch_size 个三元组，并随机构成一个错误的三元组 S'，进行更新

```

def train(self, epochs):
    nbatches = 400
    batch_size = len(self.triple_list) // nbatches
    print("batch size: ", batch_size)
    for epoch in range(epochs):
        start = time.time()
        self.loss = 0

        # Sbatch:list
        Sbatch = random.sample(self.triple_list, batch_size)
        Tbatch = []

        for triple in Sbatch:
            corrupted_triple = self.Corrrupt(triple)
            if (triple, corrupted_triple) not in Tbatch:
                Tbatch.append((triple, corrupted_triple))
        self.update_embeddings(Tbatch)

```

c) 梯度下降

定义距离 $d(x,y)$ 来表示两个向量之间的距离, 一般情况下, 我们会取 L1, 或者 L2 normal。

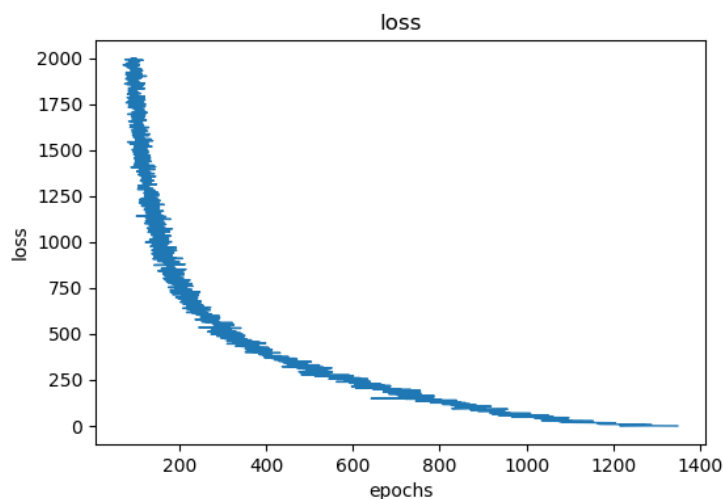
在这里, 我们需要定义一个距离, 对于正确的三元组 (h,r,t) , 距离 $d(h,r,t)$ 越小越好; 对于错误的三元组 (h',r,t') , 距离 $d(h',r,t')$ 越小越好。

$$\min \sum_{(h,r,t) \in S} \sum_{(h',r,t') \in S'} [\gamma + d(h,r,t) - d(h',r,t')]_+$$

之后, 使用梯度下降进行更新

d) 结果

选择迭代次数 2000 次, 向量维度 50, 学习率 0.01 进行训练
损失函数变化如下



结果存储在 entity_50dim1 和 relation_50dim1 中

```
transE.ipynb • transE_todo.ipynb entity_50dim1 x
entity_50dim1
1 13510 [0.2689476669044864, -0.2485949432886595, 0.33268992203297715, -0.008839313690916952, -0.06136759153078973, 0.037621910194752584, -0.23705575452674707, -0.14002512274849602,
2 8764 [0.09617177557306683, 0.09744652891237891, -0.06411287438240694, 0.2278481594177804, -0.0928666316238258, -0.054639081246656826, 0.1211938858401047, -0.043323824878830935, 0.
3 14445 [0.09850749960001509, -0.13000462156484008, 0.30182611911608415, -0.22038957868697923, -0.05107440488971577, -0.04383544719385649, 0.1981131478411786, 0.016927168095180332, 0.
4 14325 [-0.03763669822462845, -0.04942553537336966, -0.12251495674545106, -0.08584954562636886, 0.0473095765043165, 0.01336220445635528, -0.10873669878554693, 0.16783599295705076, 0.
5 12397 [-0.0122343873134392, -0.15107836346132228, 0.09155620063414475, 0.0002131467191959727, -0.03898172891701541, 0.151626081737684642, -0.12189017415181468, -0.23002805515480892,
6 8223 [-0.0723197992801677, -0.0046908545845394115, -0.1328300316504678, -0.09864012233597061, 0.16237077093379781, 0.02709990190465281, 0.15077811964108753, -0.09231675980319044,
7 14195 [-0.16824368728234977, 0.030602530812958078, -0.27918299157995997, -0.10021701650240487, -0.12385674900613006, 0.049045349198000615, 0.11618205979087812, 0.15962395071745983,
8 7144 [0.02073804225651634, 0.1685241161390486, 0.0409953917971796, -0.109060605946265743, -0.20050681380524518, -0.1103150341596406, 0.20971823014351248, -0.1624201841395525, 0.09
9 3213 [0.11543890642889927, 0.23578787199184603, -0.1790359818137922, 0.1310241625144419, -0.183004350992009783, 0.12255878286283259, 0.09289883886750736, 0.12189667392578525, -0.05
10 5178 [-0.08767504974034737, 0.026725023931436363, 0.06250737822770047, 0.09612235956594273, -0.07700041847779382, 0.22016493297439493, -0.021271639667953562, -0.07125478507627786,
11 2365 [0.036969776206447195, 0.13910157194999329, -0.20122651464641328, 0.17457147552353502, 0.032065305357619395, 0.04698300361221745, 0.00299000397843331, 0.18244523188011919, 0.
12 8654 [-0.01805645593072367, 0.2326687755878954, 0.13589152757889464, 0.10167649458176896, -0.1086150324027954, 0.10854645313777658, -0.09623112942528368, -0.17569014738348365, 0.1
582 [-0.15826536042822745, 0.14238616810755395, -0.10046969781985804, -0.002841951821669784, -0.11999542493008145, -0.06357945979464592, 0.185835987800481, 0.12487658748732479, 0.2197
14 2982 [0.07833515056551867, -0.22780237156847777, -0.0945837648702885, 0.11338630881183535, -0.03863631141288732, 0.21517107057189114, 0.034648684420785725, -0.07986834309711559, 0.
15 7508 [0.00738865823225817, 0.0027330523700939, -0.0783816710126437, 0.07069772915680957, -0.11586667445396467, -0.04709973466304191, 0.006568424641956028, -0.23013219931545104, 0
```

e) 链接预测

通过 transE 建模后，我们得到了每个实体和关系的嵌入向量，利用嵌入向量，我们可以进行知识图谱的链接预测

将三元组(head,relation,tail)记为(h,r,t)

链接预测分为三类

1. 头实体预测：(?,r,t)
2. 关系预测：(h,?,t)
3. 尾实体预测：(h,r,?)

但原理很简单，利用向量的可加性即可实现。以**(h,r,?)**的预测为例：

假设 $\mathbf{t}' = \mathbf{h} + \mathbf{r}$ ，则在所有的实体中选择与 $\mathbf{t}'$ 距离最近的向量，即为 $\mathbf{t}$ 的的预测值

f) 指标

Mean rank

对于测试集的每个三元组，以预测 tail 实体为例，我们将**(h,r,t)**中的 t 用知识图谱中的每个实体来代替，然后通过 distance(h, r, t)函数来计算距离，这样我们可以得到一系列的距離，之后按照升序将这些分数排列。

distance(h, r, t)函数值是越小越好，那么在上个排列中，排的越前越好。

现在重点来了，我们去看每个三元组中正确答案也就是真实的 t 到底能在上述序列中排多少位，比如说 t_1 排 100， t_2 排 200， t_3 排 60.....，之后对这些排名求平均，mean rank 就得到了。

Hit@10

还是按照上述进行函数值排列，然后去看每个三元组正确答案是否排在序列的前十，如果在的话就计数+1

最终 排在前十的个数/总个数 就是 Hit@10

```
14
mean rank: 353.06935721419984
hit@3: 0.12181950534103028
hit@10: 0.2754989758087725
```

g) 结果

经过 TransE 建模后，在测试集的 13584 个实体，961 个关系的 59071 个三元组中，测试结果如下：

mean rank: 353.06935721419984

hit@3: 0.12181950534103028

hit@10: 0.2754989758087725

一方面可以看出训练后的结果是有效的，但不是十分优秀，可能与 TransE 模型的局限性有关，TransE 只能处理一对一的关系，不适合一对多/多对一关系。

5. 执行 notebook 代码, 完成 TODO 任务

【实验任务】完成 notebook 文件 transe_todo.ipynb 中的内容:

(1) 补全 TODO 代码

a) 初始化实体 embedding

```
24         for entity in self.entity:
25             e_emb_temp = # 【TODO】初始化实体embedding
26             entity_dict[entity] = e_emb_temp / np.linalg.norm(e_emb_temp, ord=2)
27
```

【补全后代码如下】:

```
for entity in self.entity: # 【补全 TODO1: 初始化实体 embedding, 和关系初始化逻辑一致,
符合 TransE 论文规范】
    e_emb_temp = np.random.uniform(-6 / math.sqrt(self.embedding_dim), 6 / math.sqrt(self.embedding_dim), self.embedding_dim)
    entity_dict[entity] = e_emb_temp / np.linalg.norm(e_emb_temp, ord=2)
self.relation = relation_dict
self.entity = entity_dict
```

b) 使用范数计算正负例距离

```
124         if self.L1:
125             dist_correct = # 【TODO】使用L1范数计算正例距离
126             dist_corrupt = # 【TODO】使用L1范数计算负例距离
127         else:
128             dist_correct = # 【TODO】使用L2范数计算正例距离
129             dist_corrupt = # 【TODO】使用L2范数计算正例距离
130
```

【补全后代码如下】:

```
if self.L1: dist_correct = distanceL1(h_correct, relation, t_correct) dist_corrupt =
distanceL1(h_corrupt, relation, t_corrupt) else: dist_correct = distanceL2(h_correct, relation,
t_correct) dist_corrupt = distanceL2(h_corrupt, relation, t_corrupt) err =
self.hinge_loss(dist_correct, dist_corrupt)
```

c) 编写损失函数代码

```
181     def hinge_loss(self, dist_correct, dist_corrupt):
182         return # 【TODO】编写损失函数代码
```

【补全后代码如下】:

```
def hinge_loss(self, dist_correct, dist_corrupt): return max(0, self.margin + dist_correct -
dist_corrupt)
```

d) 计算 Mean reciprocal rank (MRR) 指标并打印输出

```
print("mean rank:", mean / len(triple_batch))
print("hit@3:", hit3 / len(triple_batch))
print("hit@10:", hit10 / len(triple_batch))
# 【TODO】计算Mean reciprocal rank (MRR) 指标并打印输出
```

【补全后代码如下】:

```
mrr += 1 / rank print(index) break print("mean rank:", mean / len(triple_batch)) print("hit@3:",
hit3 / len(triple_batch)) print("hit@10:", hit10 / len(triple_batch)) # 【补全 TODO4: 打印 MRR
指标】 print("MRR:", mrr / len(triple_batch))
```

(2) 将 notebook 每个代码段执行成功的输出截图粘贴在下面

【代码段 1 执行截图】:

```
Looking in indexes: https://mirrors.aliyun.com/pypi/simple/
Requirement already satisfied: numpy in ./anaconda3/envs/transe/lib/python3.12/site-packages (2.x.x)
Requirement already satisfied: matplotlib in ./anaconda3/envs/transe/lib/python3.12/site-packages (3.x.x)
```

【代码段 2 执行截图】:

无输出

【代码段 3 执行截图】:

无输出

【代码段 4 执行截图】:

无输出

【代码段 5 执行截图】:

```
load file...
Complete load. entity : 14951 , relation : 1345 , triple : 483142
batch size: 1207
```

【代码段 6 执行截图】:

```
epoch: 1669 cost time: 0.116
loss: 98.15850270652442
epoch: 1670 cost time: 0.12
loss: 108.63044170467023
epoch: 1671 cost time: 0.124
loss: 130.60016421514493
epoch: 1672 cost time: 0.12
loss: 98.85388160489408
epoch: 1673 cost time: 0.114
loss: 94.50546383783367
epoch: 1674 cost time: 0.113
loss: 97.07408589723386
epoch: 1675 cost time: 0.123
loss: 91.70769197704996
epoch: 1676 cost time: 0.119
loss: 87.71691318860297
epoch: 1677 cost time: 0.118
loss: 95.44174813375287
```

【代码段 7 执行截图】:

```
写入文件...
写入完成
load test file...
Complete load. entity : 13584 , relation : 961 , triple : 59071
load transE vec...
Complete load.
928
```

【代码段 8 执行截图】:

```
mean rank: 482.22
hit@3: 0.07
hit@10: 0.21
MRR: 0.06849795695055161
```

【代码段 9 执行截图】:

